

FORD VEV

Proving Ground Operational System

Guia de Implantacao e Deploy

Documentacao Tecnica - ABNT NBR 14724

Gerado em 08/06/2026

1 Prerequisites

- Node.js 18+ and npm
- Python 3.10+
- Firebase CLI (npm install -g firebase-tools)
- Firebase project access (default: ford-vev)

2 1. Data Collection App (PWA)

2.1 Local Development

```
npm install

# Install dependencies

# Start Vite dev server
npm run dev

# -> http://localhost:5173

# Enable HTTPS (for GPS on mobile)
npm run dev -- --https
```

2.2 Build

```
npm run build
```

Output in dist/:

- index.html (minified by Vite)
- assets/index-*.css (Vite-bundled CSS)
- assets/index-*.js (Vite-bundled JS - note: this is NOT used, see below)
- All 10 JS module files copied verbatim (via Vite plugin copy-static-files)
- service-worker.js, offline.html, manifest.json, icons

> Important: The app uses Firebase v9 compat SDK loaded via CDN scripts in index.html. The 10 JS files (app.js, auth-security.js, etc.) are not bundled by Vite - they are copied as-is to dist/ and loaded via <script> tags. Vite only bundles index.html and CSS into the standard assets/ folder.

2.3 Deploy to Firebase Hosting

```

# Login (one-time)
firebase login

# Deploy to production
npm run deploy
# -> https://ford-vev.web.app

# Deploy preview channel
npm run deploy:preview
# -> https://preview--ford-vev-xxxxxxx.web.app

# Deploy specific channel
firebase hosting:channel:deploy my-feature-branch

```

2.4 Hosting Config (firebase.json)

Route	Cache Policy
`assets/**`	`public, max-age=31536000, immutable` (1)
`service-worker.js`	`no-cache, no-store, must-revalidate`
`offline.html`	`no-cache, no-store, must-revalidate`
`/**` (everything else)	`no-cache`

3 2. Analytics Dashboard (Streamlit)

3.1 Local Development

```

cd dashboard

# Virtual environment
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/Mac

# Install
pip install -r requirements.txt

# Run
python -m streamlit run app.py --server.headless true
# -> http://localhost:8501

# Alternative (PowerShell):
Start-Process powershell -ArgumentList "-NoExit","-Command","python -m streamlit run dashboard/app.py --server

```

3.2 Streamlit Config

Set in `.streamlit/config.toml`:

```
[theme]
primaryColor = "#38bdf8"
backgroundColor = "#0f172a"
secondaryBackgroundColor = "#1e293b"
textColor = "#f8fafc"

[server]
headless = true
enableCORS = false
enableXsrfProtection = false
runOnSave = true
```

3.3 Production Deployment Options

Option A: Streamlit Community Cloud

1. Push dashboard/ to a GitHub repo
2. Go to share.streamlit.io
3. Connect repo, set main file to app.py
4. Add secrets (for Firebase service account):

``toml`

`.streamlit/secrets.toml` (not committed)

```
[firebase]
service_account = """
{
  "type": "service_account",
  ...
}
"""
,
```

Option B: Docker

```
# dashboard/Dockerfile

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.headless=true", "--server.port=8501"]
```

Build and run:

```
docker build -t ford-vev-dashboard dashboard/
docker run -p 8501:8501 ford-vev-dashboard
```

Option C: Windows Service

Using NSSM or scheduled task to restart on crash.

Option D: Azure / AWS VM

```
# Install Python, clone repo

cd dashboard
pip install -r requirements.txt
nohup streamlit run app.py --server.headless true --server.port 8501 &
```

1 3. Google Sheets Sync

The file Script vev tabela.txt is a Google Apps Script:

1. Go to script.google.com
2. Create new project, paste the script
3. Set the spreadsheet ID (target Google Sheet)
4. Set trigger: carregarTodasAbas() every 30 minutes (or daily)
5. The script reads from Firebase RTDB and writes to 7 sheet tabs

2 4. Firebase Service Account

The service account key is at:

```
Jsonfirebase/gestao-de-veiculos-municipal-73f568e0811b.json
```

Used by:

- Streamlit dashboard (for Firebase RTDB reads)
- Google Apps Script (via Firebase REST API)

> Security: Never commit this to public repos. The .gitignore should exclude Jsonfirebase/. For CI/CD, use environment variables or secret manager.

3 5. CI/CD

Example GitHub Actions workflow (.github/workflows/deploy.yml):

```
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-pwa:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run build
      - uses: w9jds/firebase-action@v13
        with:
          args: deploy --only hosting
    env:
      FIREBASE_TOKEN: ${ secrets.FIREBASE_TOKEN }
```

4 6. Environment Variables / Secrets

Variable	Used By	Purpose
'FIREBASE_TOKEN'	Firebase CLI	Deploy to Firebase Hosting
'GOOGLE_APPLICATION_CREDENTIALS'	Streamlit	Path to service account JSON
Firebase API key (hardcoded)	PWA	Client-side Firebase SDK

5 Quick Reference

```
# PWA: Full deploy (build + hosting)
npm run deploy

# PWA: Build only
npm run build

# Dashboard: Start
cd dashboard && .venv\Scripts\activate && python -m streamlit run app.py --server.headless true

# Dashboard: Install from scratch
cd dashboard && python -m venv .venv && .venv\Scripts\activate && pip install -r requirements.txt

# Update Firebase hosting rules only
firebase deploy --only hosting
```